

Bare Chip ESP32 Custom Development Board Hardware Design Documentation and Learning Notes V2.0

Version: V2.0

Author: Devansh Sharma (1st year Integrated MSc)

Institution: NISER, Bhubaneswar

This document covers the complete hardware design of the Bare chip ESP32 custom development board V2.0, including schematic design, PCB layout decisions, and component selection rationale.

Table of Contents

1) ESP32 Core (Processor + Support Circuit)	5
---	---

1.1) The ESP32.....	5
1.2) Power Distribution and Decoupling.....	5
1.3) Enable (EN) Pin and Reset Behaviour.....	6
1.4) Boot Strapping Configuration.....	6
1.5) Clock System (Crystal Oscillators)	7
1.5.1) 40MHz Crystal (Main Clock)	7
1.5.2) 32kHz Crystal (Optional for ESP32)	8
1.6) VDD_SDIO (Flash Power Domain)	8
1.7) Grounding and Thermal Pad.....	9
1.8) Routing Considerations.....	10
1.9) Final Verdict.....	11
2) Flash Interface (External SPI Flash)	11
2.1) Flash Interface Overview.....	12
2.2) Why Flash Design is so Critical.....	12
2.3) Routing Considerations.....	12
2.4) Pull-ups and Stability.....	13
2.5) Power Supply to Flash.....	14
2.6) Test Points for Debugging.....	14
2.7) Final Verdict.....	14
3) USB-UART Interface (USB to Serial Communication)	15
3.1) USB to UART Conversion	15
3.2) USB Signal Chain	16
3.3) Powering the USB-UART IC	16
3.4) Placement and Layout Considerations	17
3.5) Conversion of GPIO.0 pin to DTR via Silicon Lab 5.....	17
3.6) Test Points for Debugging	17
3.7) Other Control and Status Pins	17
3.8) Failure Cases	18

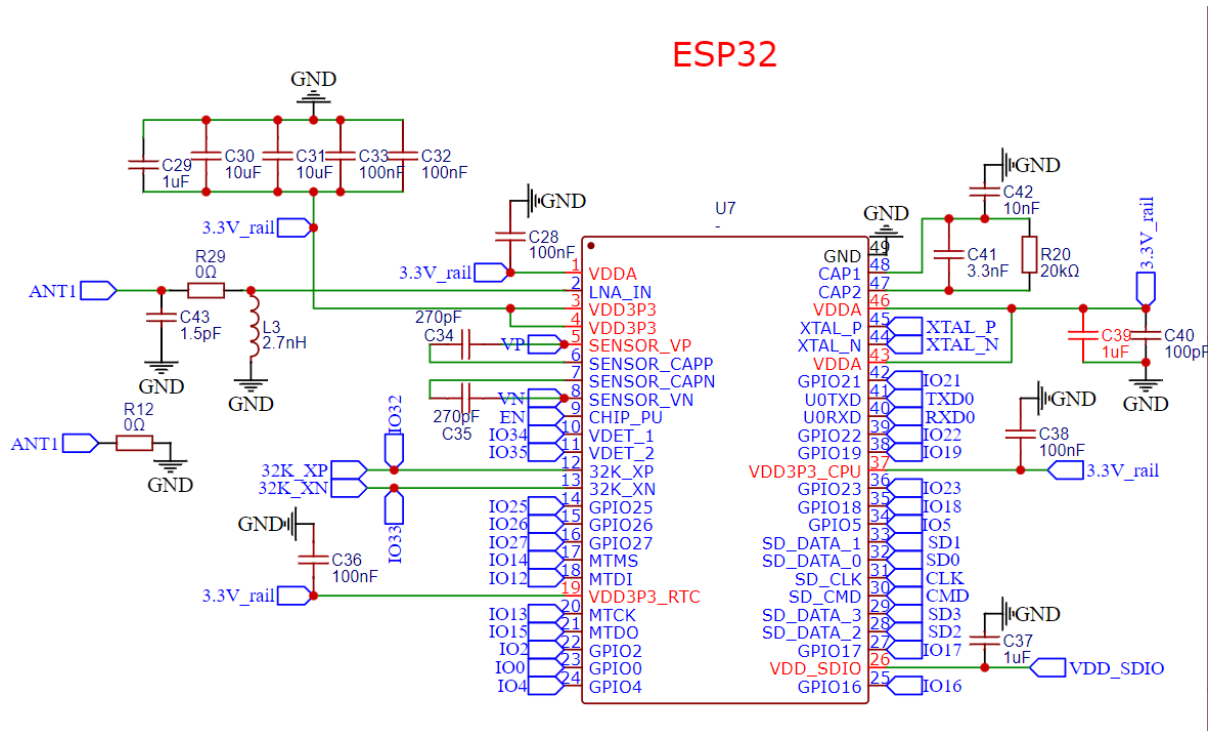
3.9) Final Verdict	18
4) Auto Reset and Boot Control Circuit	18
4.1) Transistor-Based Control	19
4.2) Working of the Circuit	19
4.3) Design and Layout Considerations	19
4.4) Test Points for Debugging	20
5) Power Block	20
5.1) Power Input Sources	20
5.2) Selection of 5V and Making of Final Rail	22
5.3) Production of Final 3.3V Rail	22
5.4) Trace Width, Power Distribution and Grounding	23
5.5) Test and Debug Support	23
5.6) Final Verdict	23
6) PCB Antenna Section (2.4GHz WIFI/Bluetooth)	24
6.1) Antenna Structure and Operation	25
6.2) RF Feed and Matching Network	25
6.3) Ground Clearance and Keep-Out Region	25
6.4) Via Stitching and Ground Reference	26
6.5) Placement Considerations	26
6.6) Layout Constraints	26
6.7) Impact of Poor Antenna Design	26
6.8) Final Verdict	27
7) Design Decisions	27
7.1) Buck Regulator over LDO for 3.3V Rail	27
7.2) CP2102N over CH340G for USB-UART	27
7.3) PCB Trace Antenna over External Module	27
7.4) LM66100 Ideal Diode over Schottky	27
7.5) External SPI Flash	28

8) Components Brief Description	28
8.1) ESP32	28
8.2) W25Q64JVSFIQ — External Flash	28
8.3) CP2102N-A02-GQFN20R — USB to UART Bridge	28
8.4) TPS62163DSGR — Buck Regulator (VIN to 5V)	28
8.5) TPS62162DSGT — Buck Regulator (5V to 3.3V)	29
8.6) LM66100DCKR — Ideal Diode Controller	29
8.7) USBLC6-2SC6 — USB ESD Protection	29
8.8) SMBJ15A — TVS Diode	29
8.9) SS14 — Schottky Diode	29
8.10) SS8050-G — NPN Transistors	30
8.11) Ferrite Bead — USB VBUS Filtering	30
8.12) Polyfuse — Overcurrent Protection	30
8.13) 40MHz Crystal — Main System Clock	30
8.14) 32.768kHz Crystal — RTC Clock	30
8.15) 270pF Capacitors — Sensor Pin Filtering	30
8.16) 5.1k Ω Resistors — USB Type-C CC Pins	31
9) Brownout Triggering	31
10) XIP (Execute-In-Place)	31
11) USB to UART Conversion (How it Actually Works)	32
12) References	33
13) Version History	33
14) Results	34

1) ESP32 core (Processor + support circuit):

This section explains the ESP32 core implementation, including power pins, decoupling, boot configuration, crystals oscillators and critical design constraint required for stable operation.

1.1) The ESP32:



The ESP32 used in this design is a highly integrated microcontroller combining digital processing, RF (WIFI and Bluetooth), and multiple internal power domains. So, unlike simpler MCUs such as ATmega328P, it is less forgiving to lose design practices. Its accurate operation depends heavily on power integrity, boot configuration, clock stability, and PCB layout design. All these things make ESP32 a system level design IC.

1.2) Power Distribution and Decoupling:

The ESP32 operates from a 3.3V supply, but internally this voltage is distributed across multiple domains through different pins:

- VDD3P3
- VDD3P3_RTC
- VDD_SDDIO

These domains serve different internal blocks (respectively) such as:

- Digital core log
- RTC subsystem
- Flash interface

Although all are connected to the same external 3.3V rail, they must be treated individually to prevent noise coupling. Each pin is decoupled separately using:

- 100nF capacitors: high-frequency noise reduction
- 1uF-10uF: transient current support

Placement rules followed:

- Capacitors are placed as close as possible to VDD pins
- Direct, short connection to ground plane using vias
- Minimal loop area

This way of layout practice is necessary for ESP32 as it exhibits dynamic current behaviour as follows:

- Low current during idle
- Sudden high current during WIFI transmission

Without proper decoupling:

- Voltage dips occur
- Brownout detector triggers, along with random resets and instability

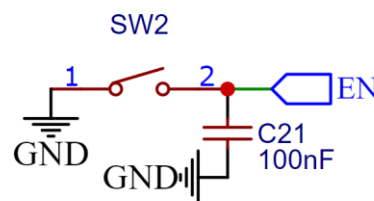
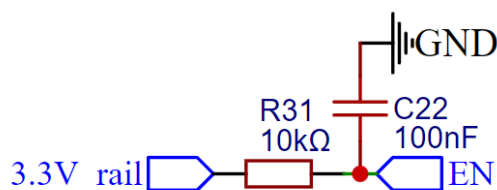
1.3) Enable (EN) Pin and Reset Behaviour:

For the ESP32 EN pin acts as the global enable/reset signal for the ESP32.

- EN must be HIGH for normal operation
- Pulled up using ~10K resistor
- Connected to auto reset circuit (later in the document) via RTS

A small capacitor (~100nF) is added between EN and GND to filter noise and providing stable power-up delay

If EN is not handled properly, then the chip may reset randomly, also startup behaviour becomes unreliable.



1.4) Boot Strapping Configuration:

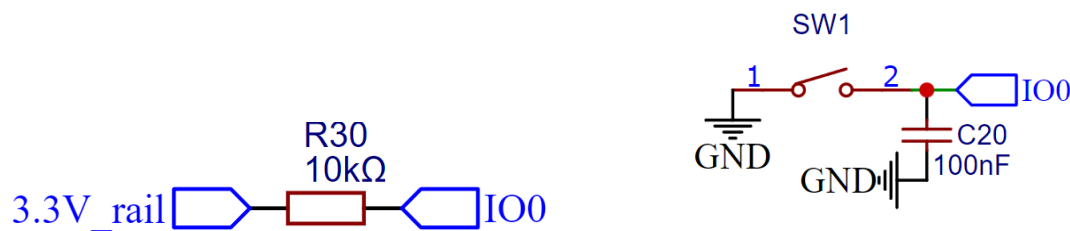
ESP32 boot mode is determined by strapping pins, by default GPIO0. The following is the boot behaviour:

- GPIO0 LOW: enters flashing mode
- GPIO0 HIGH: normal execution

What it does is, that it keeps GPIO0 pulled up by default and also controlling GPIO0 vis DTR during programming.

This ensures the automatic entry into bootloader, and normal operation after reset.

If the strapping pins are kept floating or incorrectly biased, then, boot failures occur, sometimes wrong mode gets selected, programming becomes unreliable. Ins short making entire reset system unpredictable and unreliable.



1.5) Clock System (Crystal Oscillators):

ESP32 requires a stable external clock for proper operation. Here two crystal oscillators are used:

1.5.1) 40MHz Crystal (Main Clock):

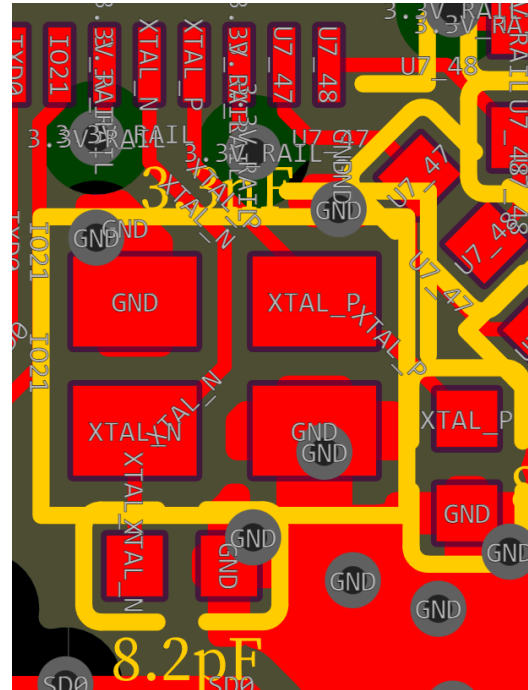
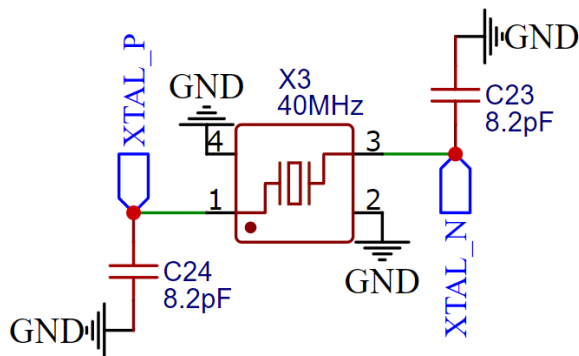
This is mandatory for CPU timing, peripheral operation, RF (WIFI/Bluetooth) functionality. It is placed very close to ESP32 pins, symmetric routing is ensured on both crystal pins (as any imbalance introduces phase error and instability in oscillation), along with two load capacitors connected to GND. Although most of the time ~18-22pF value is used as load capacitor. In my case because of different Crystal capacitance (available with me), after calculating, load capacitance came out to be ~8.2pF.

Layout rules that were followed for crystal:

- No signal routing under crystal, as it can introduce noise through capacitive coupling, which can disturb oscillation, shift frequency, make WIFI unreliable.
- Solid ground reference plane below
- Minimal loop area

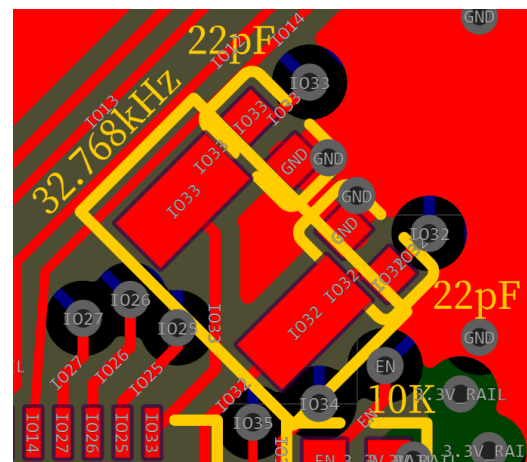
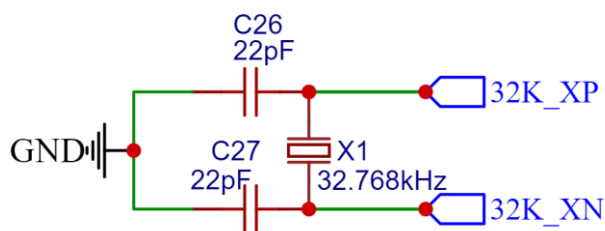
Improper crystal implementation leads to:

- Oscillation failure
- Frequency instability
- RF performance degradation



1.5.2) 32kHz Crystal (Optional for a ESP32):

Used for Low-power RTC operation. Although it is not critical for basic functionality but it improves sleep accuracy, and Timekeeping.



1.6) VDD_SDIO (Flash Power Domian):

The VDD_SDIO pin of the ESP32 is used to power external SPI flash interface. Although it can be powered by external 3.3V. but in this design, this pin is not driven directly from the main 3.3V rail. Instead, it is an output generated internally by the ESP32's on-chip regulator.

This internally regulated supply is specifically designed to power the external flash memory connected via SDIO/SPI interface. In this design, VDD_SDIO is routed from ESP32 to flash VCC pin, with local decoupling capacitors placed close to both ESP32 pin and the flash device to ensure stability.

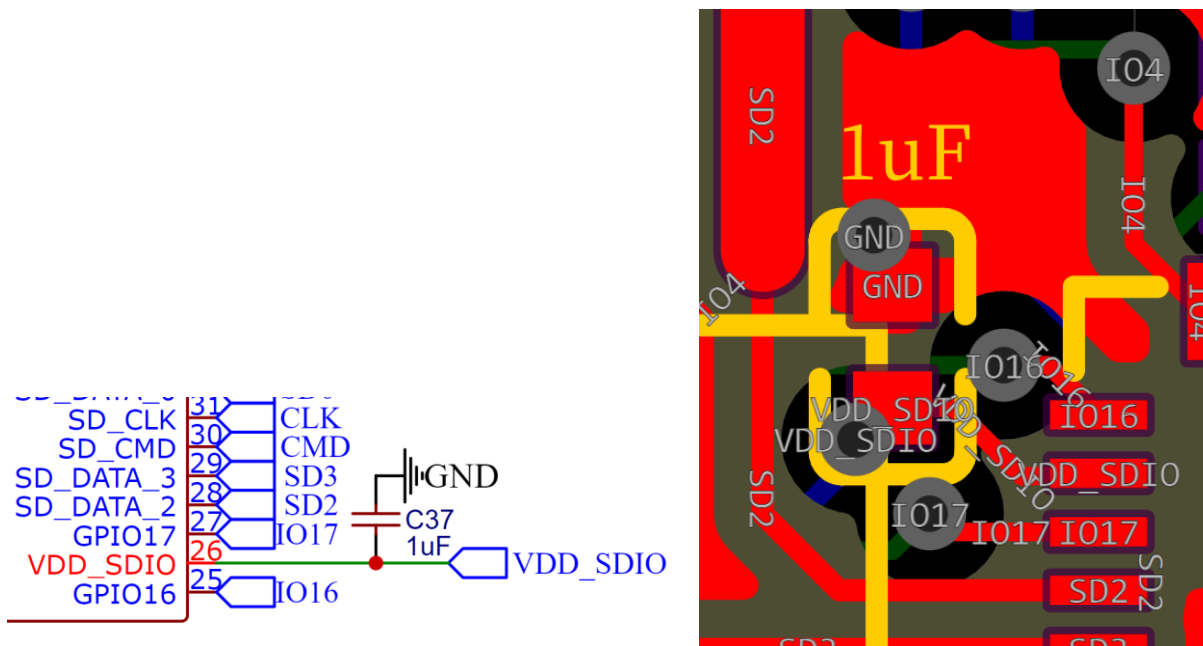
The voltage level of VDD_SDIO is determined by the ESP32 configuration (typically 3.3V in this case), and is optimized for proper operation of the flash interface. Since the ESP32 executes code directly from external flash (Execute-in-Place), this power domain becomes critical for system functionality.

If the VDD_SDIO supply is unstable or improperly decoupled, it can lead to serious system failures such as:

- Flash communication fails
- Boot failures
- Continuous resets or instability to start execution

Due to this, VDD_SDIO should be treated as a boot-critical power domain, even though it originates from within the ESP32 rather than the main power system. Proper power layout must be followed, including:

- Short routing between ESP32 and flash
- Local decoupling (100nF close to VCC of flash and also at pin of origination)
- Avoiding noise coupling from switching regulators



1.7) Grounding and Thermal Pad:

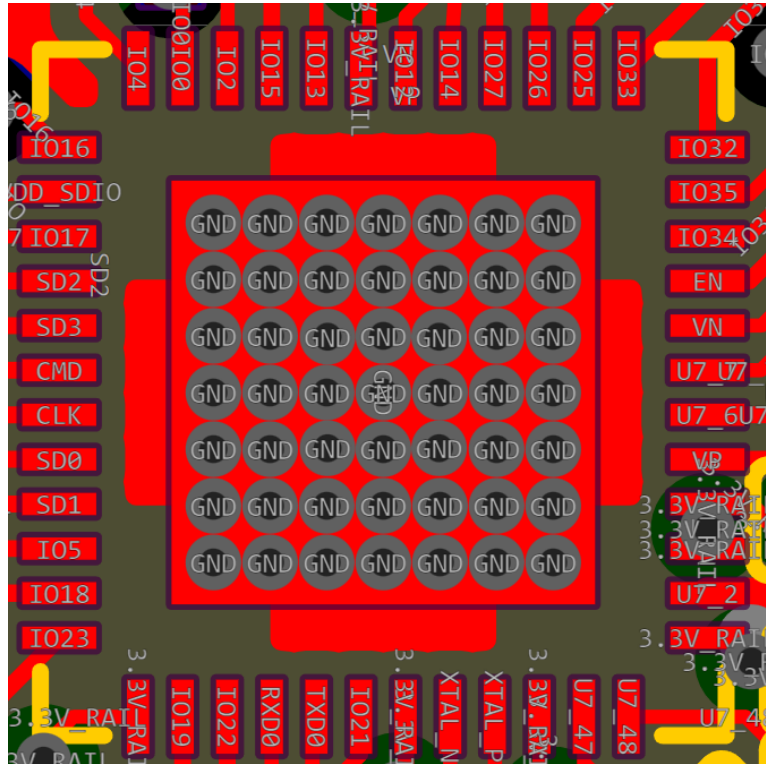
ESP32 QFN package includes an exposed thermal pad. It is connected to ground with multiple via to internal ground plane. This thing is very important as the pad serves:

- Thermal dissipation
- Central ground frame
- Noise return path

A solid ground plane is maintained under the ESP32.

This thermal pad grounding is very important as without it there is:

- Increment in impedance in return path
- Higher EMI
- Unstable RF behaviour
- Unpredictable system behaviour

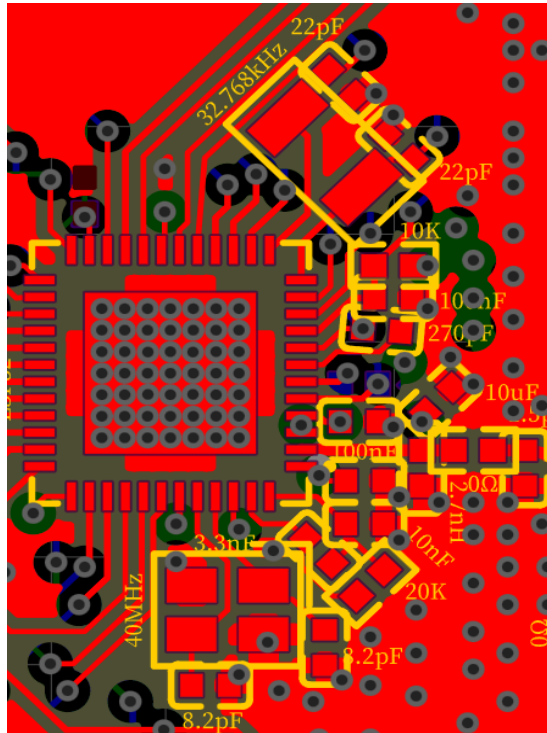


1.8) Routing considerations:

While routing signals it is:

- Avoided routing under crystal
- Avoided any routing or any copper under antenna
- Separated noisy signals from sensitive areas

This reduces interference, crosstalk, RF degradation.



1.9) Final Verdict:

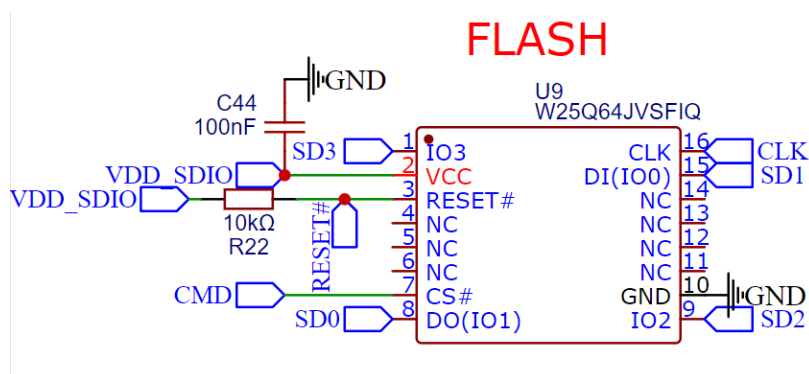
The ESP32 core is dominated by interactions between power integrity, clock stability, boot configuration, grounding. Unlike simpler designs, these factors are tightly coupled, as a mistake in one area can cause failure in another.

For example:

- Poor decoupling results in voltage dip which results in reset eventually giving boot failure
- Bad crystal results in unstable clock, which results in RF malfunction
- Incorrect GPIO0 bias which results system to be stuck in wrong mode

So, because off all these things ESP32 should be treated as a system and not just an IC.

2) Flash Interface (External SPI Flash):



The ESP32 used in this design does not contain internal flash memory. Instead, it relies on an external SPI flash for storing firmware and executing code. This makes the flash interface a critical part of the system, as any issue in this section directly prevents the ESP32 from booting.

2.1) Flash Interface Overview:

The external flash is connected using SPI (more precisely QSPI), using the following signals:

- CLK (clock)
- CS (chip select)
- IO0/IO1/IO2/IO3 (data lines) from the SD_DATA pins of ESP32

These lines form the communication link between ESP32 and flash memory. Unlike standard SPI devices, this interface operates at a relatively high speed, and is used continuously since ESP32 executes code directly from the flash.

2.2) Why flash design is so critical:

ESP32 uses execute-in-path (XIP), meaning that the code is not copied to the RAM, and is directly executed from flash. So, if flash communication is unstable then boot fails, code execution becomes unreliable and system may reset randomly.

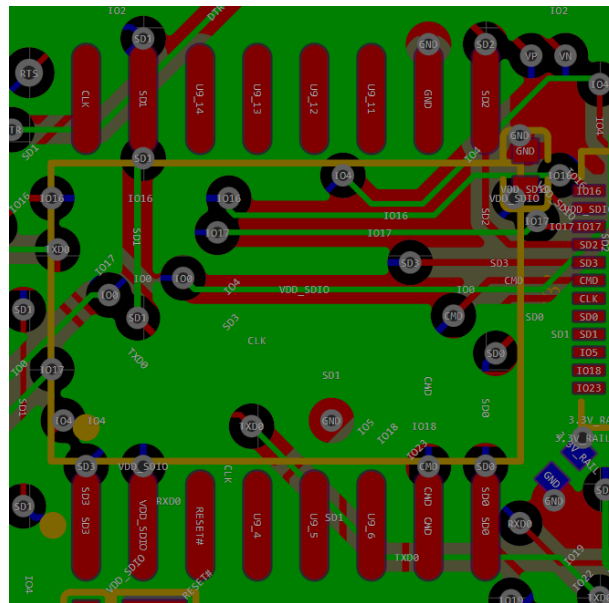
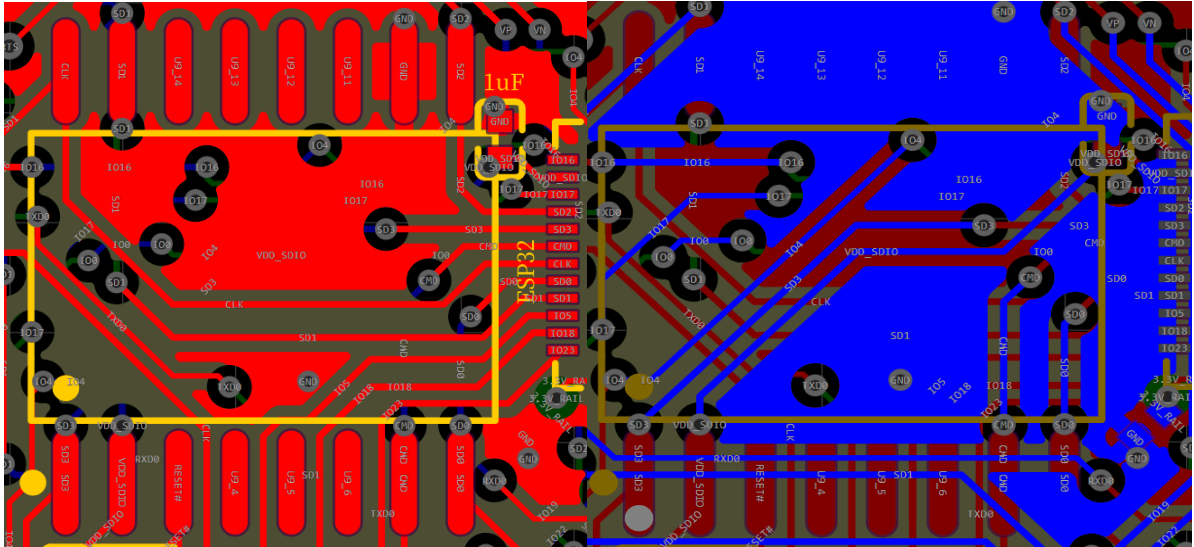
This makes flash interface boot-critical, and not just storage.

Flash is placed very close to ESP32 on the same side and with minimal obstruction, to reduce parasitic inductance and signal distortion.

2.3) Routing considerations:

Since SPI lines operate at high speed, routing must be treated carefully. The following practices were followed while routing it:

- Flash is placed very close to the ESP32
- Short and direct routing of all SPI lines
- Avoiding unnecessary vias (especially no via for CLK line)
- Similar trace lengths maintained (especially CLK and data lines), because clock and data signals must arrive with minimal skew. Otherwise, it will result in communication error and incorrect sampling of data.
- Routing is kept as short as possible as long traces introduce signal delay, ringing, and noise pickup. At high speed, even small delays can cause timing mismatch.



2.4) Pull-ups and Stability (used as per the reference design) (not used in this design):

Certain SPI lines require pull-up resistors (again, as per THE ESP32 REFERENCE DESIGN YOU ARE USING), especially for CS and IO lines (also depending upon the configuration). These ensure defined logic levels during boot.

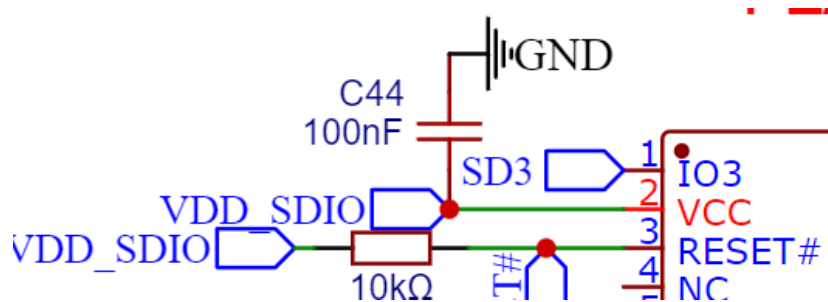
2.5) Power supply to flash:

Power supply to the flash is given through the same VDD_SDIO pin/domain of ESP32, means that it also operates at 3.3V, and is properly decoupled using a 100nF capacitor. Since ESP32 reads instructions continuously from flash, any instability here directly affects the overall function.

Not doing the above practices can result in **various kind of failures** like:

- ESP32 not booting at all
- Continuous reset loop
- Random crashes during execution
- Firmware upload issues

These failures are very difficult to debug because these are mostly embedded in PCB and hardware appears to be correct, but timing/signal integrity is wrong.



2.6) Test points for debugging:

To improve debugging and bring-up reliability, test points are provided on key SPI flash lines like CLK, CS, and IO lines. These allow:

- Probing signals using oscilloscope or logic analyser
- Verifying communication between ESP32 and flash
- Easier identification of boot failures related to flash interface

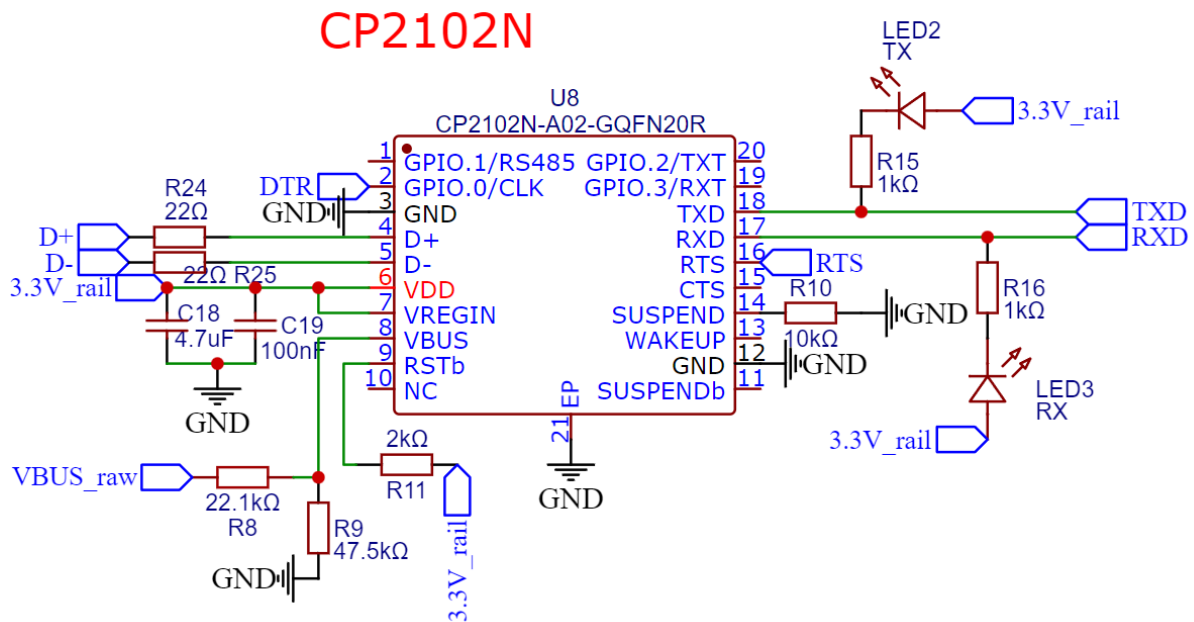
Without test points, debugging becomes difficult as direct probing on IC pins is unreliable and may disturb signal, and may also result in short-circuit if wrong pins got connected.

Test points are placed considering minimal loading and accessibility, ensuring they do not affect overall signal integrity.

2.7) Final verdict:

The flash interface in ESP32 is not just a peripheral connection. It is a part of the system. Because of this routing must be tight and clean, power must be stable, signals must be well-defined. Even small mistakes in this section can make the entire board non-functional.

3) USB-UART Interface (USB to Serial Communication):



The USB-UART interface is used to establish communication between the computer and ESP32. In this design, the 20 pin CP2102N USB to UART Bridge is used, which converts USB signals into UART signals required by ESP32.

This section is not only responsible for serial communication but also enables automatic reset and boot control during programming. So, it plays a critical role in both functionality and usability of the board.

3.1) USB to UART Conversion:

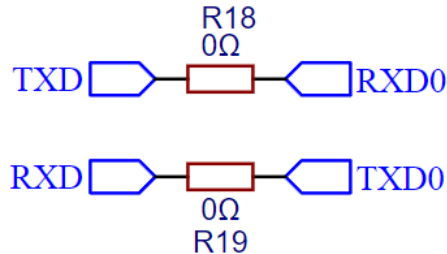
The CP2102N acts a bridge between USB (D+/D-) and ESP32 (TX/RX). In this design, the D+/D- from USB is first taken to an ESD protection diode (USBL62SC6) and to differential resistors of 22ohms to match impedance (more details in components section).

Its connections are made as follows:

- TX (CP2102N) to RX (ESP32)
- RX (CP2102N) to TX (ESP32)

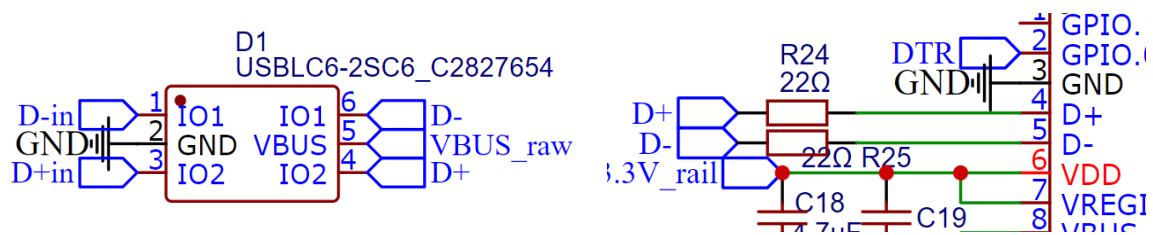
This allows serial communication, code upload, and debug logging. If this connection is incorrect then there is no communication, upload fails, and serial monitor does not work.

TRANSMISSION LINES



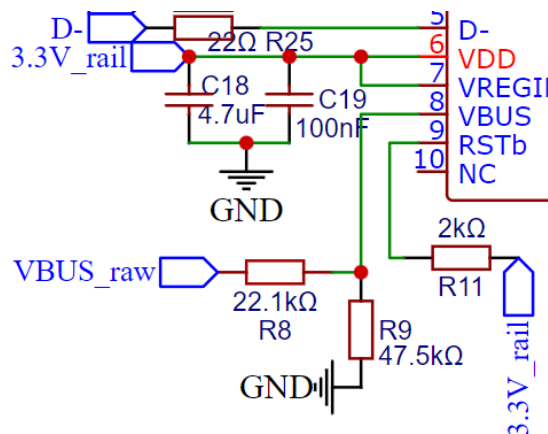
3.2) USB Signal Chain:

The signal path is from USB connector to ESD protection to Series Resistors to CP2102N. This order is important because ESD diode protects from voltage spikes, and series resistors help in signal matching with matching the impedance to ~90 ohms. If order is changed/alterd ESD can reach the IC, signal quality degrades, and USB enumeration may fail.



3.3) Powering the USB-UART IC:

In this design, CP2102N is being powered at 3.3V, or we can say with common 3.3V plane for entire board (Check the power section for detailed path of power). Poper decoupling is present at VCC pins of IC (in this design 100nF with 4.7uF). without this decoupling USB connection is unstable with random disconnects.



3.4) Placement and layout considerations:

- CP2102N is placed close to USB connector after ESD diode
- Short D+/D- routing
- UART lines routed cleanly to ESP32

This altogether reduces signal loss and noise increment.

3.5) Conversion of GPIO.0 pin to DTR via Silicon Lab 5:

Upon realising that the 20 pin CP2102N have no DTR pin. Upon discussion with ChatGPT I came to know that GPIO pins of CP2102N is configurable. So, as an experiment and knowledge gathering option, I decided to go with 20 pin CP2102N and reconfiguring its GPIO.0 as DTR in Silicon Lab 5. But this approach didn't work which is discussed later in results section.

(For consistency and clarity, the rest of this document refers to the signal as DTR, following its conventional usage.)

3.6) Test Points for debugging:

To improve debugging and bring-up reliability, test points are provided on key USB-UART and control lines: On DTR, RTS. These allows:

- Verifying UART communication between ESP32 and USB interface
- Observing reset and boot signals (DTR/RTS behaviour)
- Debugging upload failures and communication issues

Without these test points:

- Probing directly on IC pins (especially on a QFN package) is difficult and unreliable
- Signals may get disturbed during measurement
- Risk of shorting adjacent pins increases

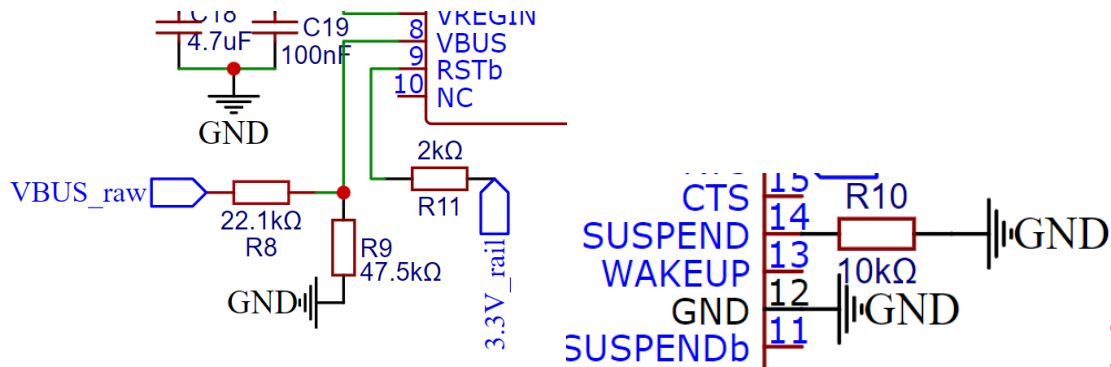
These test points are placed considering accessibility and minimal loading, ensuring that they do not affect the normal operation.

3.7) Other Control and Status Pins:

In addition to UART and reset control signals, a few auxiliary pins of the CP2102N USB to UART Bridge are used in this design to ensure proper operation and status indication:

- VBUS: connected to the USB 5V line (VBUS_raw) through a resistor divider network of 22.1k Ω and 47.5k Ω . This allows the IC to detect USB connection presence and ensure proper USB enumeration.

- VREGIN & VDD: the internal regulator pin (VREGIN) is powered from the 3.3V rail, with proper decoupling capacitors (100nF and 4.7uF). This ensures stable operation of the internal circuitry.
- RSTb: pulled HIGH using a resistor of 2kΩ to keep device in normal operating mode. Pulling this LOW would reset the USB-UART IC.
- SUSPEND: This pin indicate the USB suspend state. Although as per official reference it is connect to 10kΩ resistor, but is not used anywhere in the design.



3.8) Failures Cases:

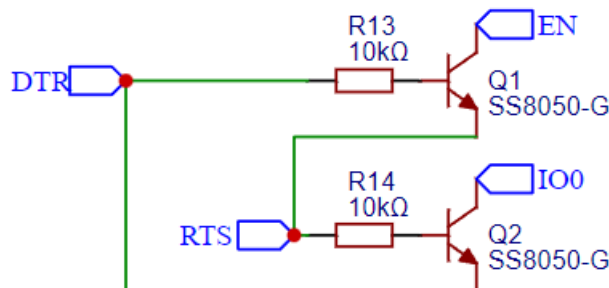
If USB-UART section is not designed properly, then errors like USB not detected, COM port not created, upload failure, auto reset failure can occur. These problems are often mistaken as software issues, while root cause is hardware design.

3.9) Final Verdict:

The USB-UART section is not just for communication but also controls the programming flow of ESP32. Correct routing, proper reset wiring, a signal integrity are essential for reliable operation. Even if ESP32 core and flash are perfectly designed, a mistake in this section can make the board unusable or development.

4) Auto Reset and Boot Control Circuit:

AUTO_RESET



The auto reset circuit is responsible for automatically placing the ESP32 into programming mode during code upload. This removes the need for manual interaction (pressing BOOT and RESET buttons) and enables seamless flashing through USB.

This circuit uses control signals from the CP2102N USB to UART Bridge, specifically DTR and RTS, along with transistor-based switching to control the ESP32's EN (reset) and GPIO0 (boot) pins. Because ESP32 enters programming mode only when GPIO0 is LOW and EN (reset) is toggled. This sequence must happen in a controlled manner during flashing. The auto reset circuit ensures this happens automatically whenever firmware upload is initiated.

In this circuitry, DTR (Data Terminal Ready) which controls GPIO0 and RTS (Request to Send) which controls EN, are the signal used. These signals are driven by USB-UART IC during the upload.

4.1) Transistor-Based Control:

Two NPN transistors (SS8050) are used in the design to control the Esp32 pins:

- One transistor for EN (reset)
- One transistor for GPIO0 (boot)

Why transistors are used:

- CP2102N outputs are logic level signals
- The ESP32 requires controlled pull-down behaviour
- Transistors provide, signal inversion, proper switching and isolation between circuits

4.2) Working of the Circuit:

During normal operation, EN is HIGH meaning that ESP32 runs normally and GPIO0 is HIGH meaning normal boot mode.

During programming, DTR pulls GPIO0 LOW, along with RTS toggling EN (reset pulse) which makes ESP32 enter into bootloader mode. After upload signals return HIGH and ESP32 runs uploaded program.

4.3) Design and Layout consideration:

- 10k pull-up resistors are used on EN and GPIO0 to ensure stable default HIGH state, along with a 100nF decoupling capacitor on EN to reduce ingoing noise
- Base resistors (10k) are used on DTR and RTS lines while connecting them to transistors to limit current into bases
- Proper routing ensuring minimal noise coupling into reset lines

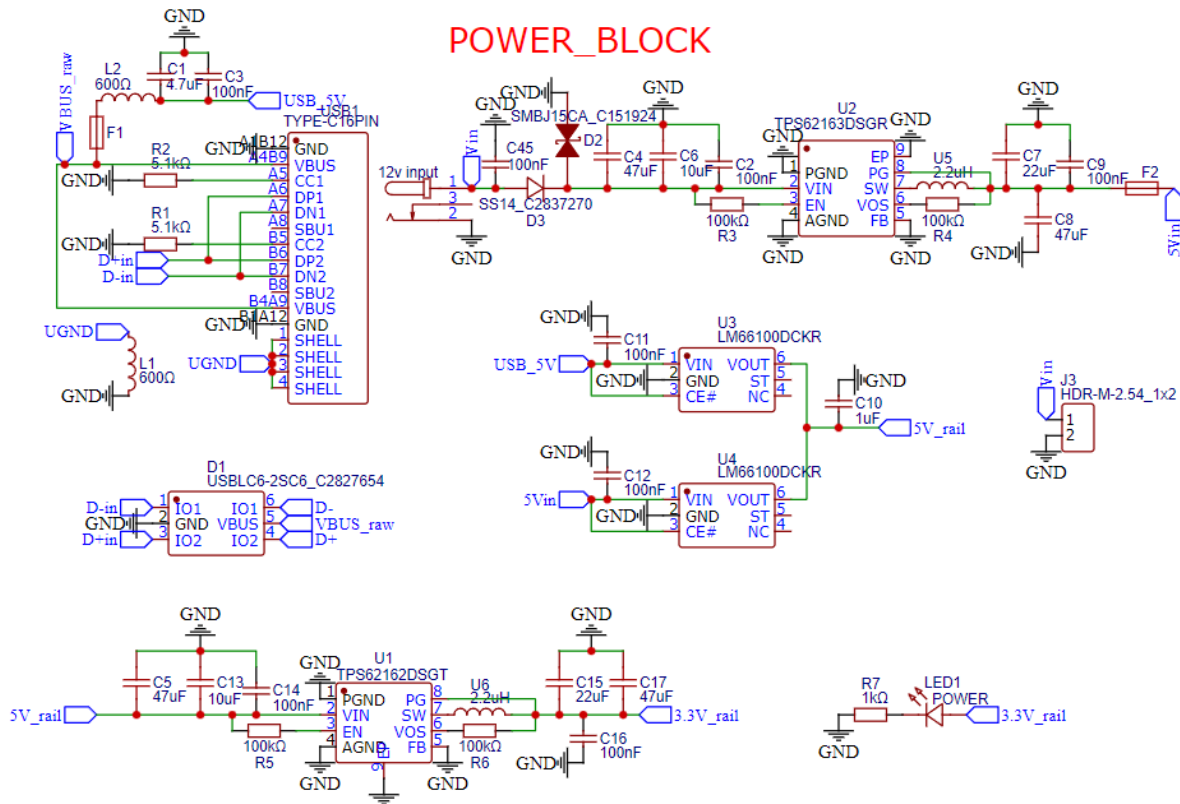
If these design considerations are not implemented:

- ESP32 does not enter flashing mode
- Upload fails repeatedly
- Manual reset may still work (indicating circuit issue)

4.4) Test points for debugging:

Test points are provided on DTR, RTS, EN, GPIO0 (GPIO0 is only on I/O pins, not provided as separate pin). This allows verifying signal transitions during upload, debugging auto-reset behaviour, and isolating issues in transistor switching.

5) Power Block:



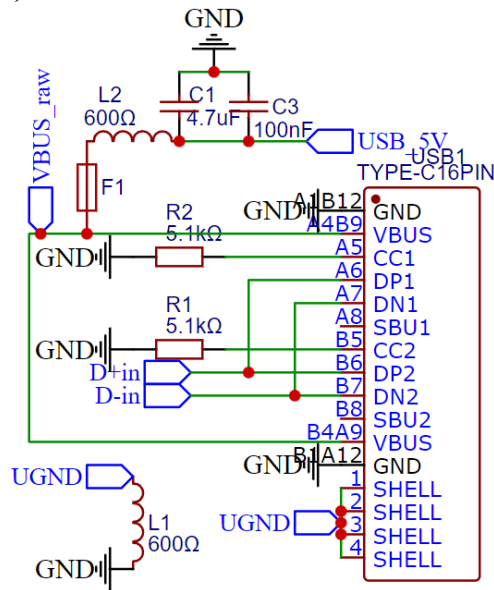
The power block is designed to accept input from multiple sources and generate stable voltage rails required for stable system operation. This design supports both USB-based power and external VIN supply, allowing flexibility with ensuring proper protection, regulation, and power integrity across the board.

The system ultimately derives a 3.3V rail, which powers the ESP32, external flash memory, USB-UART interface, and associated logic circuitry. To achieve this, a multi-stage power architecture is used, consisting of input protection, power selection, intermediate regulation, and final 3.3V conversion.

5.1) Power Input Sources:

Power can be supplied through either:

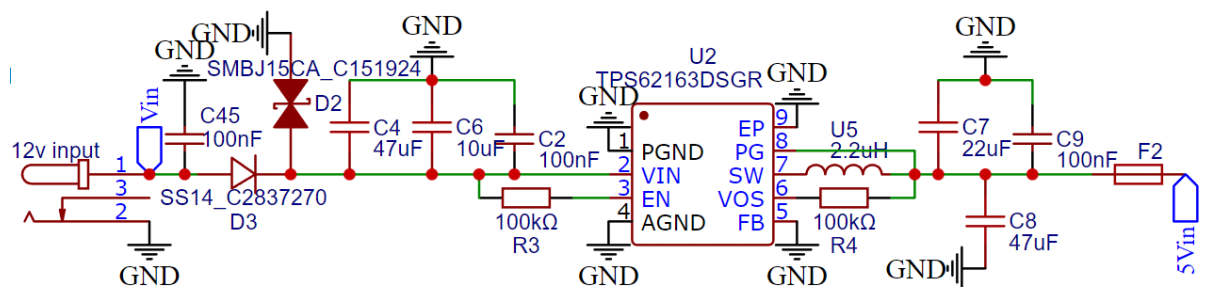
- **USB Type-C (5V VBUS):**



The USB path includes protections like a polyfuse followed by 600Ω at 100MHz ferrite bead on common VBUS line from USB-C, which protect against overcurrent conditions and surpasses high frequency noise entering from the cable to the USB-C. decoupling capacitors of 100nf and 4.7uF are used after the protection to produce 5V rail from the USB (named as USB_5V).

Also 5.1kΩ resistors are used on both CC pins to allow proper detection of USB device in PC and supplying 5V to it.

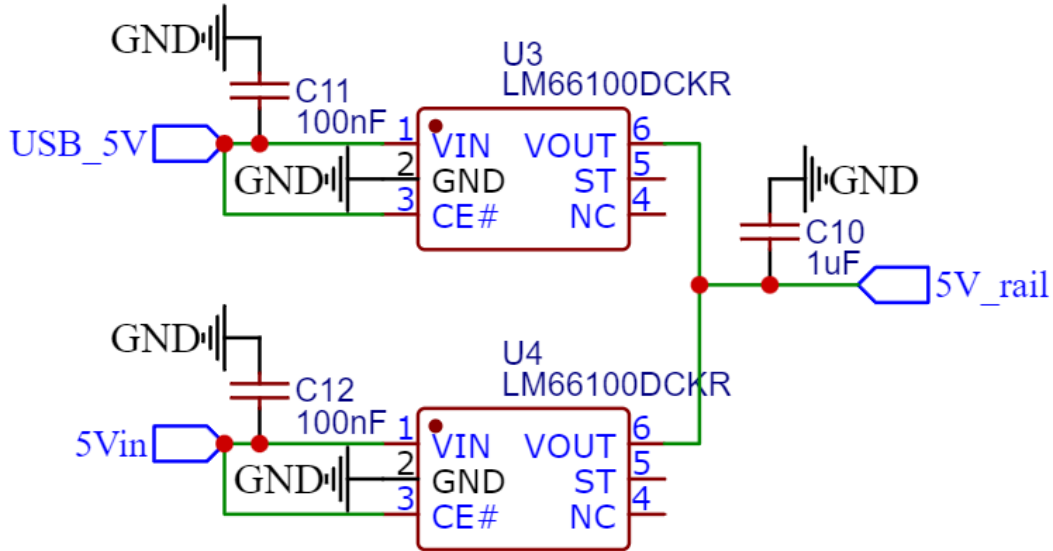
- **External VIN input (nominally higher voltage, example 12V):**



The external path is first protected by SS14 diode to prevent any case of reverse voltage going into the buck (TPS62163) along with a 100nF decoupler, followed by SMBJ15A, a bidirectional TVS ESD protection, preventing any ESD charge flowing into buck. Also, proper decoupling is also done, along with 100kΩ feedback resistor on both input and output of buck. Along with a 2.2uH inductor and proper decoupling and am polyfuse. This overall gives out 5V rail (named as 5Vin).

Here, in both input and output a 47uF bulk decoupler is also added in order to counter any current spikes during ESP32 WIFI outburst.

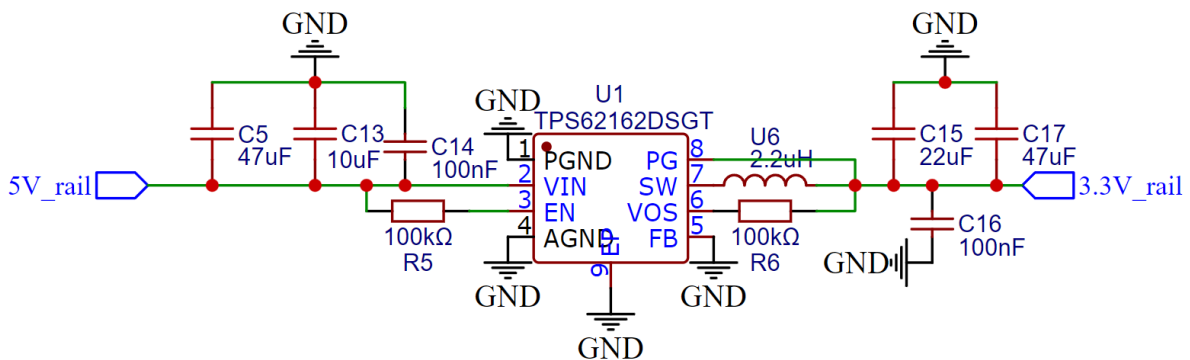
5.2) Selection of 5V and making of final rail:



Now as two different 5V rails are produced, one as USB_5V and other as 5Vin. These both 5V rails are then connected to ideal diode-ORing using two LM66100DCKR, which automatically selects the higher 5V and prevent any back-feeding of one rail into other. Thus, in this way a final 5V rail (named 5V_rail) is produced.

Also, appropriate decoupling is also done at each input and output point.

5.3) Production of final 3.3V rail:



The final 5V_rail is then passed through another buck (TPS62162), with proper decoupling at both input and output, along with a 2.2uH inductor, and on both sides a 100kΩ feedback resistor. Here no protection is added, as the 5V going in the input is well protected from start.

Also, those 47uF decouplers are also added here to counter any current spikes of ESP32 during WIFI bursts.

Although, there was a choice between a 3.3V LDO and 3.3V buck. Despite the fact, that due to fast switching of buck, extra noise may get added. I chose buck over LDO as buck are better and efficient to handle current spikes along with, using the fact that for an ESP32 development board, this minor noise addition will not be much harmful.

Decoupling and filtering is very important for the ESP32, as insufficient decoupling can result in voltage dips, leading to resets or unstable behaviour during high activity period.

5.4) Trace Width, Power Distribution and Grounding:

In all steps of power rail generation, traces are ensured to be sufficiently wide, and also it is taken care that traces are short enough to minimize resistive loss.

The regulated 3.3V is then made into a power plane (layer 3), and power to all components like ESP32, flash, etc, is given by either via to power plane at the pad edge or by a short trace and then via to power plane, it is done to make sure to ensure shortest path to allow minimal resistive loss and prevent unnecessary noise addition.

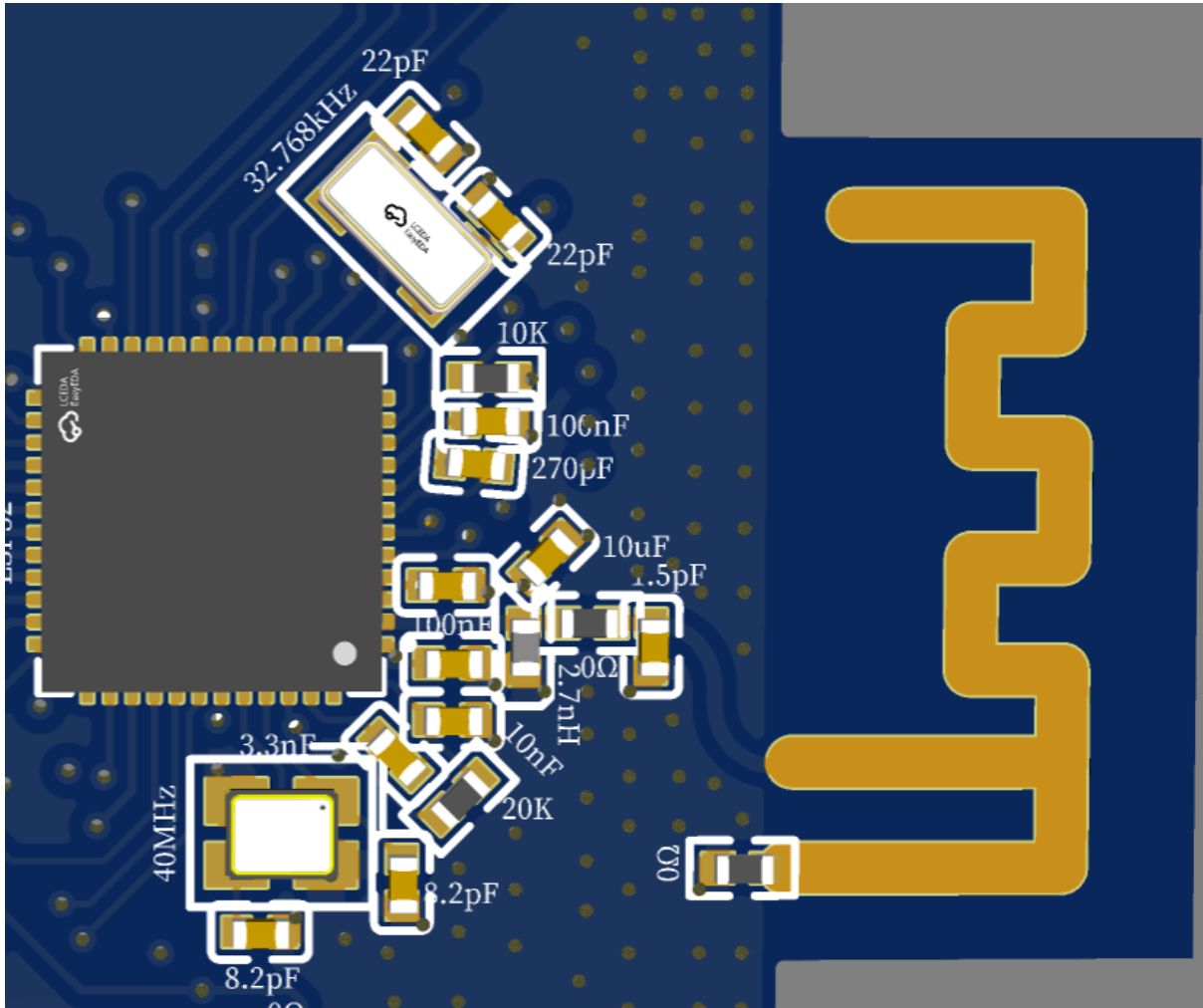
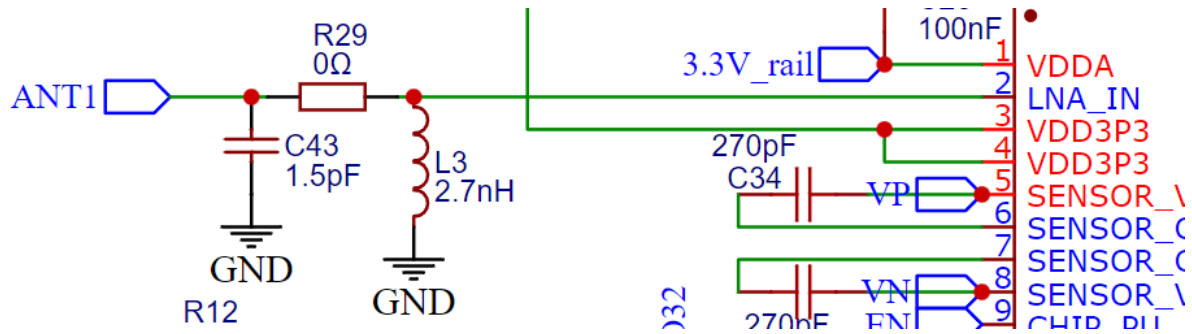
Also, a continuous GND plane is used (layer 2), which is connected to components in a same way that for power plane. This ensures the shortest possible return path for components, which in turn provides a low-impedance path, which improves overall stability and reduce noise coupling.

5.5) For Test and Debug Support, test points are provided on key power rails, including 5V_rail, 3.3V_rail, GND, Vin. This allows direct measurement of supply voltages during bring-up, and debugging, making it easier to verify regulator performance and diagnose power-related issues.

5.6) Final Verdict:

The power architecture combines efficiency, protection, and stability through a dual-stage buck conversion approach. By using switching regulators for both intermediate and final stages, the design avoids excessive power loss while maintaining reliable operation under varying load conditions. Proper filtering, grounding and layout considerations ensures that the generated rails meet the requirements of the ESP32 and associated circuitry.

6) PCB Antenna Section (2.4GHz WIFI/Bluetooth):



The ESP32 utilizes an integrated RF transceiver operating in the 2.4GHz ISM band for WIFI and Bluetooth communication. In this design, a PCB trace antenna is implemented instead of an external, providing a compact and cost-effective solution while maintaining a acceptable RF performance.

The antenna used is a meandered inverted-F antenna (MIFA), which is commonly used in ESP32-based designs due to its small footprint and reasonable efficiency.

6.1) Antenna Structure and Operation:

The antenna consists of a carefully shaped copper trace designed to resonate at approximately 2.4GHz. The meandered structure effectively increases the electrical length of the antenna while keeping the physical size small.

The antenna is fed from the ESP32 RF output through a matching network, ensuring a proper impedance matching (typically 50Ω) between the transceiver and the antenna. The following is the key characteristics of the antenna:

- Compact geometry using meandered trace
- Designed for 2.4GHz resonance
- Integrated directly on PCB (no external connector is required)
- Matched to RF front-end through passive's pi-network

6.2) RF Feed and Matching Network:



Between the ESP32 RF pin and the antenna, a matching network (here a pi network) is implemented using passive components. In this design, it includes components such as: Series element (often 0Ω or inductor placeholder) and shunt capacitors (for tuning and impedance matching). These components allow fine adjustment of the antenna impedance and resonance characteristics during design validation or future revisions.

Even if populated as default values, these components provide flexibility for RF tuning if required.

6.3) Ground Clearance and Keep-Out Region:

A critical aspect of PCB antenna performance is maintaining a proper ground clearance region around the antenna. In this design: no copper (GNP or signals or power plane) is placed directly below antenna, a defined keep-out area is maintained on all layers beneath the antenna, and ground planes and power plane is present only up to the antenna feed boundary (here 0Ω jumper).

This ensures proper radiation efficiency and prevents detuning of the antenna.

6.4) Via Stitching and Ground Reference:

A row of ground vias is placed near the antenna boundary to create a controlled transition between the RF section and the main ground plane. It provides a stable RF reference, reduces noise coupling from digital sections, and also improves isolation between antenna and the rest of circuitry.

The via fence helps contain RF currents and prevents unwanted interference from affecting antenna performance.

6.5) Placement considerations:

The antenna is positioned at the edge of the PCB, which is essential for proper radiation. The important placement decisions were made as follows:

- Located away from dense digital circuitry
- Minimal obstruction in front of antenna
- Clear space in the direction of radiation

Placing the antenna near the board edges reduces signal absorption and improves radiation efficiency.

6.6) Layout Constraints:

To ensure proper RF behaviour, the following layout practices are followed;

- Short and direct RF trace from ESP32 to antenna
- No routing of high-speed or switching signals near antenna
- Controlled impedance routing where applicable
- Decoupling capacitors placed away from antenna region

Improper layout in this region can severely degrade performance even if the schematic is correct.

6.7) Impact of Poor Antenna Design:

If antenna layout guidelines are not followed, the system may still function but with degraded performance like:

- Reduced WIFI/Bluetooth range
- Unstable connections
- Unstable packet loss
- Higher power consumption due to retransmissions

These issues are often difficult to debug because the system appears to be operational but performs poorly.

6.8) Final Verdict:

The PCB antenna implementation in this design follows the standard RF layout practices for ESP32-based systems. By maintaining proper ground clearance, controlled routing, and a defined matching network, the design ensures reliable wireless performance without requiring external antenna components.

7) Design Decisions:

7.1) Buck regulator (TPS62162) over LDO for 3.3V Rail:

A switching buck regulator was chosen over an LDO, despite the fact buck will add some noise due to high frequency switching because of the two primary reasons. First, efficiency: unlike an LDO which dissipates excess voltage as heat, a buck regulator maintains high efficiency across varying load conditions. Second, the ESP32 exhibits sudden high current demands during WIFI transmission bursts, and a buck regulator handles these transient spikes significantly better than an LDO, reducing the risk of voltage dropping and brownout triggering.

7.2) CP2102N over CH340G for USB-UART:

The CP2102N was selected as the USB-UART bridge following the official ESP32 reference design. It provides reliable auto-reset and boot control support through DTR and RTS signals, which is crucial for seamless programming without manual button interaction.

7.3) PCB Trace Antenna (MIFA) over External Module or Connector:

A meandered inverted-F antenna was implemented directly on PCB primarily as a learning exercise in RF PCB design. This approach also eliminates the need for an external antenna connector or pre-certified module, reducing cost and board complexity while providing acceptable 2.4GHz performance. Also, while giving out for fabrication the default silk screen on antenna was removed, exposing copper, improving its RF capabilities.

7.4) LM66100 Ideal Diode over Schottky for Power Selection:

The LM66100 ideal diode controller was chosen over a conventional Schottky diode or combining USB and external Vin rails. Having used it previously, its near-zero forward voltage drop makes it significantly more efficient than a Schottky, and it inherently prevents back-feeding between the two power sources, ensuring clean power selection without energy loss.

7.5) External SPI Flash:

The bare ESP32 chip does not contain any internal flash memory. External flash is therefore a hard requirement, and not a design choice, as the Esp32 executes firmware directly from external flash using execute-in-phase (XIP). The W25Q128 was selected for its adequate capacity, QSPI support, and compatibility with Esp32.

8) Components brief description:

8.1) ESP32 (Espressif Systems):

A dual-core 32-bit LX6 microcontroller with integrated WIFI (802.11b/g/n) and Bluetooth (Classic + BLE), operating at 3.3V. It does not contain internal flash and requires external SPI flash for firmware storage. Chosen as the main processing unit due to its integrated RF capability, rich peripheral set, and wide community and documentation support. Being a system level IC, it demands careful attention to power integrity, clock stability, and PCB layout for reliable operation.

8.2) W25Q64JVSFIQ (Winbond) – External Flash:

A 64Mbit (8MB) serial NOR flash memory supporting standard, dual, and quad SPI (QSPI) interface. Used as the sole firmware storage for the ESP32, which executes code directly from it using execute-in-phase (XIP). Chosen for its adequate capacity, QSPI support for high-speed access, and full compatibility with ESP32 flash interface requirements. Since ESP32 reads instructions continuously from this device, it is treated as a boot-critical component.

8.3) CP2102N-A02-GQFN20R (Silicon Labs) – USB to UART Bridge:

A 28 pin USB to UART bridge IC that converts USB signals to UART for serial communication with ESP32. Chosen for its reliable auto-reset and boot control support through DTR and RTS signals, enabling seamless automatic programming without manual button interaction. Provides stable driver support across all major operating systems.

8.4) TPS62163DSGR (Texas Instruments) – Buck Regulator (Vin to 5V):

A high-efficient synchronous step-down buck regulator used to convert the external Vin supply (nominally 12V) down to 5V. chosen for its high efficiency, compact package, and ability to handle the input voltage range expected from external supply. Operates with a 2.2uH inductor, 100k Ω feedback resistor and appropriate decoupling to produce a clean 5V rail (5Vin) from the external path.

8.5) TPS62162DSGT (Texas Instruments) – Buck Regulator (5V to 3.3V):

a high-efficiency synchronous step-down buck regulator used to convert the final 5V rail down to 3.3V rail for powering the entire board. A buck was chosen over an LDO for two reasons, higher efficiency across varying loads, and superior handling of sudden high current demands during ESP32 WIFI transmission bursts, reducing the risk of voltage drop and brownout triggering. Operating with a 2.2uH inductor, 100kΩ feedback resistors and 47uF bulk decoupling to counter current spikes.

8.6) LM66100DCKR (Texas Instruments) – Ideal Diode Controller (×2):

An ideal diode controller with near zero forward voltage drop, used in pairs for power rail ORing between USB_5V and 5Vin rails. Automatically selects the higher available 5V source while preventing back-feeding from one rail into the other. Chosen over a Schottky diode due to its significant lower voltage drop and active back-feed prevention, resulting in more efficient and reliable power selection.

8.7) USBLC6-2SC6 (STMicroelectronics) – USB ESD Protection:

A low capacitance ESD protection diode array specially designed for USB signal lines. Placed at the USB input before the CP2102N to protect the IC from electrostatic discharge events on the USB connector. Its low capacitance ensures minimal impact on USB signal integrity, which is critical for reliable USB enumeration and communication.

8.8) SMBJ15A – TVS Diode (Vin Protection):

A bidirectional transient voltage suppression diode placed on the external Vin input path. Clamps any ESD or voltage transient spikes before they can even reach the buck regulator (TPS62163), protecting the downstream circuitry. The 15V clamping voltage is selected to be above the maximum expected input voltage while still providing effective protection.

8.9) SS14 – Schottky Diode (Reverse Polarity Protection):

A Schottky diode placed on the external VIN input to prevent reverse voltage from entering the buck regulator in case of incorrect power supply connection. Chosen for its low forward voltage drop and fast switching characteristics, minimizing power loss in the protection path.

8.10) SS8050-G (x2) – NPN Transistors (Auto Reset Circuit):

NPN transistors used in the auto reset and boot control circuit to switch the ESP32's EN and GPIO0 pins based on DTR and RTS signals from CP2102N. Transistors are used instead of direct connection because the CP2102N outputs are logic level signals, while the ESP32 pins require controlled pull-down behaviour with proper isolation. Provides signal inversion and clean switching needed for reliable automatic programming.

8.11) Ferrite Bead (600Ω at 100MHz) – USB VBUS Filtering:

A ferrite bead placed on the VBUS line from the USB connector. Suppresses high frequency noise entering from the USB cable into the board power path, acting as a low-pass filter for the power line. The 600Ω impedance at 100MHz ensures effective attenuation of switching and cable-induced noise while passing DC without significant loss.

8.12) Polyfuse – Overcurrent Protection:

A resettable polyfuse placed on the USB VBUS path for overcurrent protection. Limits current in case of a short circuit or excessive load, protecting both the USB host and board circuitry. Automatically resets after the fault is removed, unlike a one-time fuse.

8.13) 40MHz Crystal – Main System Clock:

Used as the primary clock source for the ESP32, mandatory for CPU operation, peripheral timing, and RF (Wi-Fi/Bluetooth) functionality. Placed very close to the ESP32 crystal pins with symmetric routing and 8.2pF load capacitors, calculated based on the specific crystal's load capacitance specification. Improper implementation directly affects RF performance and system stability.

8.14) 32.768kHz Crystal – RTC Clock:

An optional low-frequency crystal used for the ESP32's real-time clock (RTC) subsystem. Not critical for basic operation but improves sleep mode timing accuracy and timekeeping. Connected with 22pF load capacitors as per standard RTC crystal requirements.

8.15) 270pF Capacitors – ESP32 Sensor Pin Filtering (SENSOR_VP, SENSOR_VN):

The ESP32's SENSOR_VP and SENSOR_VN pins are connected to the internal ADC and are sensitive to noise. 270pF capacitors are placed on these pins as per the official ESP32 hardware design reference. They act as low-pass filters, suppressing high frequency noise on these sensitive analog input pins and improving ADC measurement stability. Although included as per reference design, their function is genuine and important for analog signal integrity.

8.16) 5.1kΩ Resistors – USB Type-C CC Pins:

Two 5.1kΩ resistors placed on the CC1 and CC2 pins of the USB Type-C connector. Required for proper USB device detection as without these resistors, the USB host cannot identify the board as a valid USB device and will not supply 5V VBUS power. This is a mandatory requirement for USB Type-C device implementation.

9) Brownout Triggering:

ESP32 has an internal voltage monitor. If supply voltage drops below a threshold (around 2.5-2.7V), even momentarily, ESP32 automatically resets itself. This is called brownout reset. It's a protection mechanism as running at insufficient voltage causes unpredictable behaviour, so it's safer to just reset.

During WIFI transmission, current spikes can cause momentary voltage dip on the rail. If decoupling is insufficient, that dip crosses the brownout threshold which results in automatic reset overall resulting in random crashing.

That's why decoupling and buck over LDO matters directly for this.

10) XIP (Execute-In-Place):

Normally in a microcontroller, firmware is copied from flash into RAM first and then executed from RAM. This requires RAM to be large enough to hold the entire program.

XIP is different. The processor reads and executes instructions directly from flash memory, without copying them to RAM first. This means:

- RAM is freed up for actual data and variables
- Larger programs can run than RAM alone would allow
- Flash becomes part of the execution path, not just storage

This is why the flash interface is boot-critical in ESP32. If flash communication is unstable even for a moment, instruction fetch fails, and the processor crashes. It's not just reading stored data; it's actively running code from there in real time.

11) USB to UART Conversion (How it actually works) and why CP2102N is needed:

The CP2102N acts as a protocol bridge between two fundamentally different communication standards, USB and UART.

USB does not transmit data as simple logic signals. Instead, it uses differential signalling on two lines, D+ and D-. Data is encoded as the difference between these two lines, when D+ is high and D- is low it represents one logic state, and when D- is high and D+ is low it represents the other. This differential approach makes USB inherently noise resistant and capable of operating at high speeds. Additionally, USB is a packet-based protocol, meaning data is wrapped in structured packets with handshaking, error checking, and device enumeration, making it significantly more complex than simple serial communication.

The ESP32 on the other hand communicates via UART, a simple two wire protocol (TX/RX) where data is transmitted as plain voltage levels at a fixed baud rate. High voltage is logic 1, low voltage is logic 0. No differential signalling, no packets, no handshaking.

These two protocols are fundamentally incompatible and cannot communicate directly.

The CP2102N internally contains two separate engines:

USB Engine: Receives the D+/D- differential signals, decodes the USB protocol, handles enumeration with the host computer, extracts the raw data bytes from USB packets, and manages all USB complexity internally. The computer simply sees it as a standard virtual COM port.

UART Engine: Takes the extracted data bytes from the USB engine and transmits them as simple UART signals on the TX pin to the ESP32 and vice versa receives UART data from ESP32 on RX pin and packages it back into USB format for the computer.

These two engines are internally connected inside the CP2102N, making the entire protocol translation invisible to both sides. The computer sees a USB device, the ESP32 sees plain UART. Neither needs to know about the other's protocol.

This is why a USB-UART bridge IC is necessary, without it, the computer has no way to communicate with the ESP32, making firmware upload, serial monitoring, and debug logging impossible.

In this design, the D+/D- lines from the USB Type-C connector are first passed through the USBLC6-2SC6 ESD protection diode, then through 22Ω series resistors for impedance matching to approximately 90Ω differential impedance, and finally into the CP2102N. This order is critical, ESD protection must come before the IC to protect it from voltage spikes, and series resistors ensure signal integrity for reliable USB enumeration and communication.

12) References:

- 1) ESP32 Official Schematic diagram:
https://dl.espressif.com/dl/schematics/esp32_devkitc_v4-sch.pdf
- 2) ESP32-WROOM-32 Datasheet:
https://documentation.espressif.com/esp32-wroom-32_datasheet_en.pdf
- 3) TPS62162 and TPS62163 Datasheet:
<https://www.ti.com/lit/ds/symlink/tps62160.pdf>
- 4) LM66100 Datasheet:
<https://www.ti.com/lit/ds/symlink/lm66100.pdf?ts=1776595592249>
- 5) CP2102N Datasheet:
<https://www.silabs.com/documents/public/data-sheets/cp2102n-datasheet.pdf>
- 6) USBLC6-2SC6 Datasheet;
<https://www.st.com/resource/en/datasheet/usblc6-2.pdf>
- 7) W25Q64JVSFIQ Datasheet:
https://www.mouser.com/datasheet/2/949/w25q128jv_revf_03272018_plus-1489608.pdf?srltid=AfmBOoocx1SUbsPEsFNV4bSN0bcbfBeo2O1kVCMpfmXbO174Q6DXkjQt
- 8) For components like Crystals and anything else, Datasheet from its LCSC is used while placing components in EasyEDA

13) Version History:

Version	History
V1.0	The initial schematic contained a minor error, as DTR and IO0 logic nets got connected in auto-reset transistor as they were initially connected to same 10k ohm pull-up
V2.0	Corrected schematic mistakes from V1.0. completed PCB layout with improved routing width, as using 5mil near ESP32 signals and transitioning to 6mil or higher (as required) as soon as space is available, instead of just 5mil for full signal, improving reliability for manufacturing. Also, it was made sure that the silk-screen on PCB is removed before fabrication.

14) Results:

- Auto reset and boot of the board is basically useless as GPIO.0 of CP2102N was not supported to be configurable to DTR (not only GPIO.0, but no pin of 20 pin CP2102N was configurable to DTR as per Silicon Lab 5, which removed the option of bypassing the circuit if any other pin is configurable)
- On manual reset and boot board works as normally as an ESP32 board is expected to work
- Board works without any power wake-up issue, which was an issue in my ESP32 board which was made using ESP32 WROOM 32D module
- The antenna of board works perfectly as it can detect networks from range of -7dBm to -96dBm
- Board operates on DIO + flash frequency set to 40MHz, but struggled to work on QIO + flash frequency to 80MHz, which emphasis on the scope on improvement of flash routing